

AMANAT

Amount-contingent settlement for agent-initiated payments

"We would rather refuse our own mechanism than ship a capability we could not cite."

Eeshan Singh Pokharia

eeshan.singh53@gmail.com · github.com/HERPESME/amanat

THE PROBLEM

An agent pays before the price exists.

"Book a cab to the airport – cap it at ₹1,000."

₹620

the ceiling the agent must commit —
before the meter runs

0 retries 90 days

if the ceiling is low — the debit dies;
NPCI grants none for this decline

if it is high — the customer's money can
sit blocked for nothing

A human watches the meter and pays what it says. An agent is asked to forecast — on rails built for known prices.

WHAT THE MARKET LACKS

Everyone proves permission. Then stops.

STANDARD	WHAT IT PROVES	WHERE IT STOPS
Google AP2	a signed mandate — the agent may spend	authorization
OpenAI / Stripe ACP	checkout of a known amount	prepay • fixed price
Coinbase x402	a machine paid per call	prepay • no contingency
Visa TAP • MC Agent Pay	who the agent is	identity only

GAP 1 settle an amount that does not exist yet — ceiling now, actual later.

GAP 2 “my agent did it” — no post-transaction record to settle a dispute against.

Gap 2 in the industry's own words — the card networks and dispute processors flag the post-authorization layer as unaddressed.

THE TRAP

Razorpay already runs agentic payments.

Reserve Pay pilots are live — Zomato, Swiggy, Zepto.

So we did not build another checkout.

Checkout is solved. What is not solved is settling an amount that does not exist yet — and proving afterwards, to someone who does not trust you, what the agent's money actually did.

The interesting problem is not moving the money. It is the receipt.

The unit of trust: a signed transition.

A transition carries

- the proposal the model's ask, verbatim
- the verdict deterministic — and it cites its reason
- the rail state block · debit · release · refuse
- an Ed25519 signature over the entry's bytes
- a hash link to the entry before it

Three consequences

Refusal is evidence.

The times it said no are in the record, signed — the half ordinary logs leave out.

The model holds no authority.

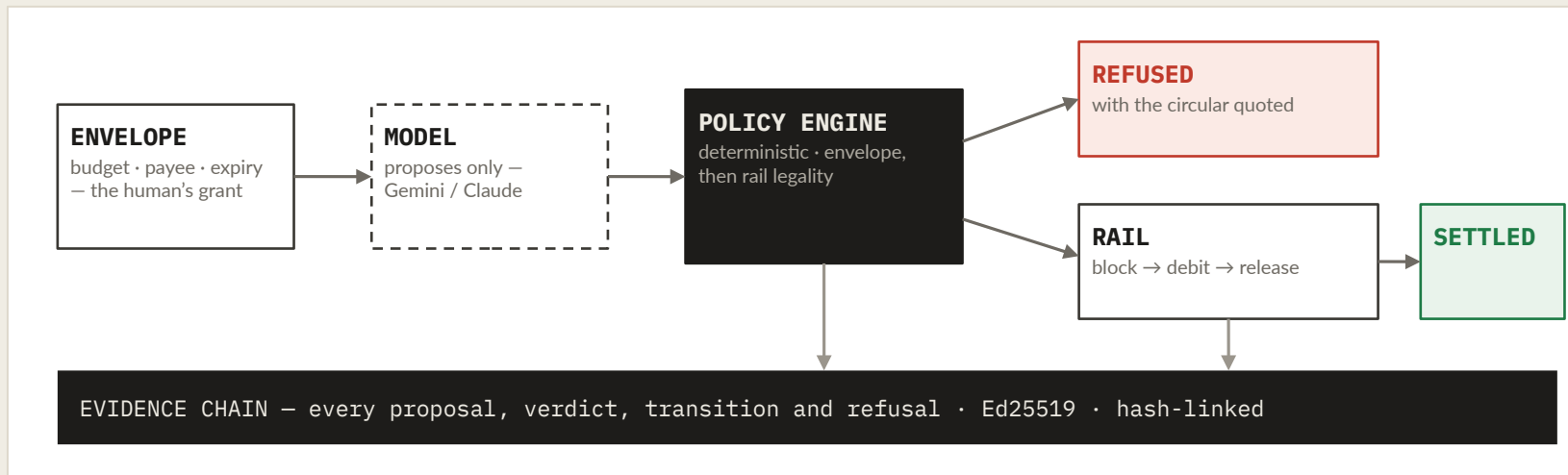
Gemini or Claude, swappable. A wild prompt yields refusals, not payments.

Anyone can verify.

The packet re-checks its own hashes and signatures in your browser — zero trust in us.

HOW IT WORKS

Propose. Dispose. Enforce. Record.



The LLM proposes. The policy engine disposes. The rail enforces.

No path reaches money without a verdict — pinned by test, then held over 600+ random action sequences per CI run.

Invariants proven property-based, not by example: debited + released ≤ blocked · blocked ≤ budget · available ≥ 0. No counterexample found.

EVIDENCE DISCIPLINE

The gate held – against us.

For one day, it refused its own mechanism

Partial debit rested on vendor docs. Tier UNVERIFIED – so permits() returned False, for us too. Absence of evidence is not permission.

```
SourceTier.UNVERIFIED → permits() == False
```

Then we read the circular

NPCI/UPI/OC-228 – image-only scans, read page by page. Partial debit holds by necessary implication; tier PRIMARY, quote attached. The gate opened.

“The current block limits (unutilised) are always checked before initiating a debit.”

Five evidence tiers, one rule. PRIMARY (circulars) · OBSERVED (the live API's own response) · SECONDARY (PSP docs) are usable as fact. MARKETING and UNVERIFIED never are – 26 capabilities across 5 rails, every one cited or refused.

The day of refusal is preserved in the suite as a record, not deleted – the gate keying off evidence, not convenience, is the design.

MEASURED, NOT QUOTED

A doc says should. A probe says did.

PROBE	RESULT	WHAT IT MEANS
Razorpay • partial capture	HTTP 400	“Capture amount must be equal to the amount authorized” — the live rail and the docs agree word for word.
Setu UMAP • reachability	NXDOMAIN	credentials accepted (HTTP 200) — but uatapi.setu.co / api.setu.co resolve on neither Google nor Cloudflare DNS.
Razorpay • live settlement	rfnd_TT5K...	capture ₹620 → refund ₹150 → merchant nets ₹470 — a real refund id, inside the signed chain.
NPCI OC-228 • block cap	₹10,000	read from the scanned circular, enforced in code — a reserve above it is refused with the quote attached.

Reproduce: `python -m amanat.rails.probe` • `python -m amanat.rails.settle` — test mode throughout; no real money moved.

THE ADD-ON

"My agent did it." Check the record.

✓ SUPPORTS MERCHANT

"The ₹470 charged was authorized and within every bound" — cites the mandate at entry #0, the debit at entry #8.

● CHARGE NOT IN CHAIN

"The disputed ₹5,000 was never charged — that attempt was refused at entry #2." Refusals are evidence.

✗ OUTSIDE EVIDENCE

"The signed record cannot establish delivery." The honest limit, stated — not papered over.

An evidence finding — not an issuer decision. It states what the signed record shows. No win-rate is claimed — win-rate is the issuer's call, not the record's. The export is a one-click representment packet: authorization + evidence + finding.

THE NUMBERS

Every figure matches the runner.

205

tests — all pass with zero credentials and zero network

23

adversarial attacks — homoglyphs, overflows, sequence abuse — all refused

600+

random action sequences per CI run — money invariants held after every step

26

rail capabilities across 5 rails — every one cited, or refused

The ceiling model, honestly. Conformal quantile regression on 300k real NYC metered fares. Finding: the coverage guarantee is distribution-free but not shift-free — 18 of 20 configs missed nominal under a real temporal split; recency calibration narrows the gap $\sim 10\times$ ($-3.55pp \rightarrow +0.91pp$). A random split would have passed — and lied about deployment.

RUNNING IT

Try to make it misbehave.

amanat-demo-699979063196.asia-south1.run.app

Attack the envelope → watch the refusal, with its citation → dispute the charge → download the representment packet. Real governed core; bring-your-own-key live Gemini agent; the receipt re-verifies in your browser.

```
python -m amanat.demo
    the seven-act walkthrough
python -m amanat.compare
    two rails, side by side
```

```
python -m amanat.dispute.demo
    settle on AP2, then contest it
python -m amanat.rails.settle
    a real run on Razorpay test APIs
```

Cloud Run, scales to zero — it costs nothing idle; open it once beforehand to skip the cold start.

STATED BEFORE YOU ASK

What this is not.

The SBMD demo runs on a simulator.

Every semantic cites the circular it models; the real-rail run is gated on PSP enablement. The real Razorpay path is the different-but-honest capture-then-refund.

Consent binding is a placeholder.

Adjudication proves the settlement conformed to the recorded grant — not that a human's key signed it. AP2's cnf slot is where that lands.

No risk model of ours.

Razorpay ships RTO Shield. Risk enters through a seam, not a rewrite.

No win-rate, no “first”, no invented data.

Negative findings are promoted, not buried. The adversarial review that killed our own v1 thesis ships in the repo.

**Agents will transact before trust does.
Amanat is the trust part.**

Eeshan Singh Pokharia · eeshan.singh53@gmail.com · github.com/HERPESME/amanat